Turtlzim

Suivi d'avancement du projet

état au 16 octobre 2026 — dernier commit `ce7bd15+`

À quoi sert ce document

Turtlzim est une application de notes dont les notes sont au format wiki de Zim, avec chiffrement par carnet, en trois versions (web, bureau, mobile) qui partagent un seul code. Ce document garde la trace de **ce qui a été fait, ce qui est en cours, en attente, reporté ou écarté, et ce qui reste à faire pour chaque point**. Il est repris et recompilé à chaque étape, pour qu'on puisse relire d'un coup d'œil où l'on en est. Les chiffres de tests et les mots « vérifié » ou « non vérifié » ne disent que ce qui a réellement été exécuté ; ce qui n'a jamais été essayé est dit comme tel (section 5).

Légende

FAIT	Fait et vérifié par des tests (le « reste à faire » dit ce qui manque encore autour).
EN COURS	Commencé, pas terminé.
EN ATTENTE	Prévu, pas commencé : il manque du temps, un outil, une réponse de votre part ou une autre étape.
REPORTÉ	Laissé pour plus tard, volontairement : ça a du sens, mais pas maintenant.
ÉCARTÉ	Abandonné ou remplacé, avec la raison : on n'y reviendra pas sans nouvelle décision.

Synthèse

FAIT	28	EN COURS	1	EN ATTENTE	9
REPORTÉ	3	ÉCARTÉ	8		

Dernier relevé des tests (au 16 octobre 2026) : `cargo test` du format, des règles du carnet et de leurs liaisons wasm : 297 réussis (9 ignorés, qui demandent le corpus de Zim) ; tests unitaires de l'application web : 555 réussis (5 ignorés) ; tests du serveur de comptes : 96 réussis ; scénarios dans un vrai navigateur : 123 sur 123(Chromium) et 107 sur 107(WebKit, le moteur d'un iPhone).

1 Où on en est, point par point

N°	Point	État	Où on en est	Reste à faire
A — Fondations (format, cœur, port WebAssembly)				
A1	**Format Zim : lecture et écriture**	FAIT	Un lecteur/écrivain du format Zim (`core/zim-format`), réécrit sans copier le code de Zim. Un fichier relu puis réécrit est identique octet pour octet : test d'aléa à chaque `cargo test`, et test sur de vrais carnets de Zim (`scripts/zim-golden.sh`). Deux défauts trouvés par ces tests ont été corrigés.	Aucune relecture extérieure. Le corpus d'essai est celui que Zim fournit : d'autres carnets réels pourraient montrer des cas rares.
A2	**Cœur Turtl en Rust remis en état**	FAIT	Le cœur de Turtl (dormant, outils anciens) compile et se teste avec les outils actuels (édition 2018, async partout). Documenté dans `docs/core-rs-revival-notes.md`.	Mettre à jour la libsodium d'origine du build natif (la 1.0.16 est notée dans `CLAUDE.md` ; à vérifier où elle sert encore).
A3	**Port WebAssembly du cœur**	FAIT	Chiffrement par la vraie libsodium (libsodium.js, mêmes octets que le natif, vérifié), SQLite réel compilé en wasm, réseau asynchrone, stockage persistant (OPFS), le tout dans un Worker.	Rien de connu côté web. Le pendant natif pour les téléphones est le point D5.

N°	Point	État	Où on en est	Reste à faire
A4	**Règles du carnet**	FAIT	Noms, liens, arbre, rétroliens, recherche, renommage qui garde les liens, étiquettes, journal, complétion des liens, tableaux : un seul endroit (`core/zim-notebook`, `zim-format`), appelé par l'application via wasm.	Comparer plus largement avec le comportement de Zim sur des carnets réels (aujourd'hui : corpus de Zim seulement).
B — Application web				
B1	**Éditeur (CodeMirror 6)**	FAIT	Mise en forme vivante du texte Zim (le document *est* le fichier, jamais un arbre réécrit), raccourcis, tableaux en grille, images, en-tête repliée.	Fidélité complète à l'éditeur de Zim (risque 7 de `CLAUDE.md`) : syntaxes rares et réglages de Zim non reproduits.
B2	**Carnet : pages, liens, recherche**	FAIT	Arbre des pages, rétroliens, recherche, renommer/supprimer, import de dossier ou de zip, export en zip.	—
B3	**Pièces jointes et images**	FAIT	Joindre, glisser-déposer, coller ; images dessinées dans la page ; une revue a trouvé et fait corriger un SVG qui s'exécutait.	—
B4	**Journal, étiquettes, sommaire, tableaux**	FAIT	Calendrier du journal, panneau d'étiquettes, sommaire de la page, complétion de `[[liens]]`, tableaux.	—
B5	**Écriture sur téléphone**	FAIT	Barre de mise en forme, appuis tactiles, mise en page qui suit le clavier ; testé avec des événements tactiles et un clavier simulé, en Chromium et en WebKit.	**Jamais essayé sur un vrai téléphone** (seulement l'émulateur Android) : le comportement du clavier réel peut différer.
B6	**Application installable, hors ligne, code épinglé**	FAIT	Service worker qui liste chaque fichier avec son empreinte et ne lance que la version approuvée ; mises à jour qui attendent l'accord.	Le worker lui-même vient de l'hébergeur : un hôte hostile n'est pas arrêté, seulement rendu vérifiable (risque 6). Le paquet de bureau est la garantie la plus forte.
C — Chiffrement local				
C1	**Carnets chiffrés (coffre)**	FAIT	Cœur Turtl en wasm dans un Worker, une phrase secrète (scrypt, 128 Mio), un compte local, un « espace » par carnet ; rien de lisible ailleurs que dans le coffre (un test scanne tout ce que garde le navigateur).	Mesurer le déverrouillage sur de vrais téléphones (181 s sur l'émulateur Android à deux cœurs émulés).
C2	**Changer la phrase, sauvegarder, restaurer**	FAIT	Changement de phrase repris après un arrêt en cours de route ; sauvegarde du coffre en un seul fichier toujours chiffré ; restauration dans un autre navigateur ; coupures testées à plusieurs moments.	Une sauvegarde ne suit pas un changement de phrase (documenté) ; clés des carnets non renouvelées par un changement de phrase.
C3	**Un seul chemin de chiffrement**	ÉCARTÉ	Règle tenue partout : libsodium seule, jamais un second code de chiffrement par plateforme.	Écarté : tout chiffrement refait à la main ou par plateforme.
D — Applications (bureau, mobile)				
D1	**Bureau Windows et Linux (Tauri 2)**	FAIT	La même application web dans Tauri ; paquets `.deb`, `.rpm`, `.AppImage`, NSIS et MSI construits par GitHub Actions à chaque changement.	macOS non fait ; signature des installeurs ; mise à jour automatique ; **jamais lancé sur une vraie machine Windows** (la CI seulement).
D2	**Carnets en vrais dossiers (bureau)**	FAIT	Un carnet est un dossier Zim lu et écrit sur place, changement tout ou rien avec journal de reprise ; Zim peut ouvrir le même dossier.	Modification simultanée avec Zim lui-même : non essayée.
D3	**Android (Capacitor)**	FAIT	APK de débogage, essayé dans une vraie WebView sur émulateur (douze vérifications automatiques) ; aucune permission réseau, pas de sauvegarde automatique.	Signature de publication, boutique, `appId` provisoire, essai sur téléphone réel ; la permission `INTERNET` sera nécessaire pour les comptes (point E17).
D4	**iPhone / iPad**	EN ATTENTE	Rien de commencé : le même principe (Capacitor) s'applique, WebKit est déjà testé.	Il faut un Mac avec Xcode (ou un macOS en CI), puis un essai sur appareil.

N°	Point	État	Où on en est	Reste à faire
D5	**Cœur natif relié par JNI / FFI**	REPORTÉ	Les coques font tourner le cœur wasm dans leur WebView : ça marche et évite deux chemins de code.	À reprendre si le coffre est trop lent sur téléphone (point C1) ; le dépôt de référence `turtl/mobile` montre la liaison Android.
D6	**Application Flutter / Dart**	ÉCARTÉ	Question posée le 13 octobre : non, l'éditeur serait à réécrire (plusieurs semaines, deux éditeurs à garder identiques).	Écarté. À rouvrir seulement si l'éditeur dans une WebView s'avérait inacceptable sur appareil réel.
D7	**Interfaces natives GTK / Qt**	ÉCARTÉ	Décision du 5 octobre : une interface web partagée plutôt que trois interfaces.	Écarté.
E — Comptes, serveur et sauvegardes en ligne				
E1	**Étude de faisabilité**	FAIT	Étude en français (`docs/zim-cloud-feasibility.md`) : hébergement, comptes, connecteurs, chiffrement, feuille de route en lots 0 à 8, 9 décisions ouvertes, risques.	Les 9 décisions ouvertes (point E18).
E2	**Hébergement mutualisé OVH**	ÉCARTÉ	Lot 0 de l'étude (essai d'hébergement) abandonné le 15 octobre : on continue sur ce serveur.	Écarté pour l'instant ; un VPS sera acheté « si le projet aboutit » (point E3).
E3	**Serveur de comptes sur cette machine**	FAIT	`server/` : Node 24, SQLite intégré, sert l'application et `/api/`. Tourne sur le port 8443 (https avec certificat local).	Mise en service pour d'autres personnes : VPS, nom de domaine, https de confiance, sauvegarde de la base, supervision, mises à jour. Rien de tout cela n'est fait.
E4	**Serveur de synchro d'origine de Turtl**	ÉCARTÉ	Remplacé de fait par `server/` (Node 24 + SQLite, protocole à nous) ; l'ancien (Node 8 + Postgres) n'a pas été revu.	Écarté ; `CLAUDE.md` le cite encore parmi les « décisions verrouillées » : à corriger avec votre accord.
E5	**Comptes par e-mail, code à l'inscription**	FAIT	Inscription (adresse + mot de passe + code à 6 chiffres envoyé une seule fois), connexion, mot de passe oublié, export des données, suppression ; mots de passe en scrypt, sessions en cookie `HttpOnly`, limites de tentatives, protections contre le vol de session et l'énumération.	**Envoi réel d'e-mails jamais essayé** (sans serveur SMTP, les codes vont dans un fichier) ; SPF/DKIM ; changer le mot de passe depuis l'interface, « se déconnecter partout », vue d'administration.
E6	**Code à chaque connexion**	ÉCARTÉ	Votre décision du 15 octobre : le code seulement à l'inscription.	Écarté.
E7	**Connexion Google et GitHub**	FAIT	Flux OAuth complet côté serveur (secret jamais dans la page, PKCE, état à usage unique, liaison au navigateur, adresses confirmées par le service seulement).	**Jamais essayé avec les vrais services** : à vous de créer les deux applications OAuth (étapes dans `docs/zim-server.md`), puis un essai réel.
E8	**Facebook, LinkedIn, Microsoft, GitLab (connexion)**	REPORTÉ	Décision du 15 octobre : après Google et GitHub.	Un pilote par service, même schéma qu'aujourd'hui ; une application OAuth à créer chez chacun.
E9	**Synchronisation en clair vers le compte (1 Mo)**	FAIT	« Garder ce carnet sur le serveur » : envoi automatique, récupération, question quand les deux côtés ont changé (jamais d'écrasement), refus d'une écriture faite depuis une ancienne révision ; essai à deux navigateurs avec le vrai serveur.	Fusion page par page (aujourd'hui une copie remplace l'autre) ; envoi du seul changement (le carnet entier part à chaque fois) ; quota par carnet ; les carnets de dossier du bureau et le téléphone ne se synchronisent pas encore.
E10	**Copie scellée des carnets chiffrés**	FAIT	La sauvegarde du coffre (déjà chiffrée) peut être envoyée au serveur : rien de lisible n'en sort (un test examine chaque requête et le fichier stocké) ; restauration sur un navigateur sans coffre.	Manuelle et sans fusion. Synchronisation chiffrée fichier par fichier, avec hiérarchie de clés et clé de secours : demande de nouvelles commandes dans le cœur Rust (points E16a à E16d).
E11	**Connecteur Nextcloud / ownCloud / WebDAV**	FAIT	Le serveur appelle le service pour vous (mot de passe d'application chiffré au repos, adresses privées refusées, aucune redirection suivie) ; seuls les fichiers changés sont écrits ; essayé contre un faux serveur WebDAV.	**Jamais essayé avec un vrai Nextcloud**. Écriture non atomique sur WebDAV.

N°	Point	État	Où on en est	Reste à faire
E12	**Connecteur GitHub**	FAIT	Un envoi = un commit (API Git Data), branche déplacée sans force : deux appareils ne s'écrasent pas ; essayé contre un faux GitHub.	**Jamais essayé avec le vrai GitHub** (limites de débit, branches protégées, commits signés exigés).
E13	**Connecteur GitLab**	FAIT	Un envoi = un commit avec `last_commit_id`, listes paginées ; essayé contre un faux GitLab.	**Jamais essayé avec un vrai GitLab** ; la détection d'un conflit repose sur les mots de GitLab.
E14	**Connecteur Dropbox**	FAIT	Autorisation dans la fenêtre de Dropbox (le serveur ne voit jamais le mot de passe), dossier d'application, jeton de rafraîchissement chiffré, révoqué quand on déconnecte ; envois par révision (un fichier modifié ailleurs n'est pas écrasé) ; essayé contre un faux Dropbox.	À vous : créer l'application Dropbox (`docs/zim-server.md`) ; **jamais essayé avec le vrai Dropbox**. Pas de validation en une fois sur plusieurs fichiers (limite de Dropbox).
E15	**Connecteur Google Drive**	FAIT	Autorisation dans la fenêtre de Google, portée la plus étroite (`drive.file`), dossier « Zim Notebooks », vrais dossiers Drive ; essayé contre un faux Drive.	À vous : créer l'application Google ; attention, en mode « Test » Google arrête le jeton après 7 jours (à vérifier chez Google pour la publication) ; **jamais essayé avec le vrai Drive**.
E16a	**Synchro par page des carnets en clair**	EN COURS	Étape 1 de la synchronisation fichier par fichier, décidée le 16 octobre : on compare, page par page, ce que chaque côté a changé depuis le dernier accord ; seules les pages changées sont envoyées ou récupérées ; deux appareils qui changent des pages différentes se fusionnent seuls ; une même page changée des deux côtés est demandée (garder les deux, garder la mienne, prendre l'autre).	Tout : index par page sur le serveur et les connecteurs, fusion, fenêtre de conflit, essais à deux appareils (l'étape est commencée, rien n'est encore livré).
E16b	**Couche de chiffrement du cœur Rust**	EN ATTENTE	Conçue : une clé par carnet, enveloppée par la *phrase du coffre local* (réutilisée, décision du 16 octobre : pas de seconde phrase à taper) ; un objet scellé par page et par pièce jointe (XChaCha20-Poly1305), noms masqués, index scellé.	Nouvelles commandes dans le cœur (`LOCAL_COMMANDS`), vecteurs de test communs natif/wasm, essais ; décider d'une clé de secours (aujourd'hui : phrase perdue = carnets perdus, comme le coffre).
E16c	**Pastilles et création en deux choix**	EN ATTENTE	Décidé : une seule liste de carnets, avec deux pastilles à mot (« Chiffré », « Synchronisé (où) » ou « Cet appareil seulement ») et une icône d'état ; la fenêtre de création demande les deux axes ensemble.	Écrire l'interface et ses essais (captures, scénarios) ; regroupement par destination au-delà d'une dizaine de carnets.
E16d	**Carnets chiffrés vers le serveur et les connecteurs**	EN ATTENTE	Même format d'objets scellés partout : sur Nextcloud, GitHub, GitLab, Dropbox ou Drive, un carnet chiffré n'est qu'un dossier d'objets illisibles.	Dépend de E16b et E16a ; ouverture sur un nouvel appareil (le carnet apparaît verrouillé, la phrase du coffre est demandée une fois).
E16e	**Déplacer une page vers un autre carnet**	EN ATTENTE	Réponse au besoin « une page secrète parmi des pages ordinaires » : un second carnet chiffré, et un moyen d'y passer une page.	Fonction à écrire (liens entrants à signaler, pièces jointes à emporter).
E16f	**Notion de « projet » (regroupement de carnets)**	REPORTÉ	Décision du 16 octobre : l'unité reste le carnet ; une notion de projet pourra s'ajouter si le besoin se confirme.	À reprendre plus tard : regrouper plusieurs carnets, chacun gardant son chiffrement et son emplacement.
E16g	**Carnet mixte clair/chiffré**	ÉCARTÉ	Un carnet est soit chiffré, soit en clair, jamais les deux : les liens, rétroliens, étiquettes, recherche et arbre relient toutes les pages, et une page chiffrée à côté de pages en clair se trahirait par ces index.	Écarté ; pourrait être repris plus tard si la pertinence est démontrée.
E16h	**Carnet mixte local/distant**	ÉCARTÉ	Un carnet est soit local, soit distant (synchronisé), jamais les deux de façon durable ; il ne l'est mélangé que pendant un changement (envoi initial, changement d'emplacement, conflit en cours).	Écarté comme état durable ; extension possible plus tard.

N°	Point	État	Où on en est	Reste à faire
E17	**Connexion au compte depuis les applications natives**	EN ATTENTE	Rien de commencé.	Connexion par adresse locale avec PKCE (bureau), permission `INTERNET` (Android), changement de la politique de contenu des coques.
E18	**Décisions ouvertes de l'étude (§10)**	EN ATTENTE	Chiffrement proposé par défaut ou non ; une seule destination par carnet ; une ou deux variantes des applications natives ; éditeur légal ; sens du quota (1 Mo) ; gratuit ou payant.	Vos réponses. Aucune n'a été donnée ; j'ai suivi les valeurs par défaut proposées (en clair d'abord, une destination par carnet).
E19	**Cadre légal**	EN ATTENTE	Rien : éditeur, conditions d'utilisation, politique de confidentialité, RGPD (les données sont déjà exportables et supprimables).	Texte à rédiger et à faire relire avant toute ouverture au public.
E20	**Durcissement avant ouverture au public**	EN ATTENTE	Protections de base écrites et testées (limites, cookies, en-têtes, adresses privées refusées). Aucune revue de sécurité extérieure.	Revue extérieure, journalisation et alertes, sauvegardes du serveur, essais de charge, procédure de mise à jour, plan en cas de fuite.
F — Méthode et outils				
F1	**Intégration continue (bureau)**	FAIT	GitHub Actions construit les paquets Linux et Windows à chaque changement ; nouvelles tentatives si un téléchargement échoue.	—
F2	**Ce document de suivi**	FAIT	Source `docs/suivi/suivi.tex`, PDF compilé par `scripts/suivi.sh` (tectonic installé sans root).	Le tenir à jour à chaque étape (section suivante).

2 Principes d'organisation des carnets

Décidés le 16 octobre 2026, avec le vocabulaire : **carnet** (l'unité), **page** (une note ; une page peut avoir des sous-pages, ce qui donne les sections), **pièce jointe**. Ces règles sont des *choix*, pas des limites techniques : elles pourront être assouplies plus tard si l'usage le justifie.

Axe	Règle retenue	Extension possible plus tard
Chiffrement	Un carnet est **soit chiffré, soit en clair** : il ne contient jamais à la fois des pages chiffrées et des pages en clair. L'unité de chiffrement est le carnet (une clé par carnet).	Carnet mixte (quelques pages chiffrées dans un carnet en clair), si on trouve comment ne rien révéler par les liens, rétroliens, étiquettes, recherche et arbre. Aujourd'hui : un second carnet chiffré et une fonction « déplacer la page » (E16e).
Emplacement	Un carnet est **soit local** (seulement sur cet appareil), **soit distant** (synchronisé : compte du serveur ou service connecté), une seule destination à la fois. Il ne mélange pas de pages locales et de pages distantes, **sauf temporairement** pendant un changement (envoi initial, changement d'emplacement, conflit en cours de règlement).	Carnets dont certaines pages ne quittent jamais l'appareil ; plusieurs destinations à la fois pour un carnet.
Stockage et synchronisation	Un carnet distant est stocké **page par page** (un objet par page et par pièce jointe) avec un index ; seules les pages changées voyagent ; deux appareils qui changent des pages différentes se fusionnent seuls. Un carnet chiffré suit le même schéma, avec des objets scellés.	Découpage plus fin à l'intérieur d'une page (fusion de lignes) ; compactage de l'index pour de très gros carnets.
Phrase secrète	Les carnets chiffrés (locaux ou distants) réutilisent **la phrase du coffre local** : une seule phrase, jamais redemandée pour la même session. Une phrase perdue = carnets perdus (aucune clé de secours aujourd'hui).	Clé de secours imprimée une fois ; phrase propre à chaque carnet.
Interface	Une **liste unique** de carnets ; deux pastilles avec un mot : « Chiffré » et « Synchronisé (où) » ou « Cet appareil seulement » ; une icône d'état (à jour, envoi en cours, conflit, erreur).	Regroupement par destination au-delà d'une dizaine de carnets ; regroupement de carnets en « projets » (E16f).

3 Décisions prises

Date	Décision	Conséquence
5 oct. 2026	Une interface web partagée (CodeMirror 6) pour le web, le bureau et le mobile.	Pas d'interfaces natives par plateforme (point D7).
6 oct. 2026	Carnets chiffrés par le cœur Turtl en wasm, dans un Worker, un compte local par phrase secrète.	Base de tout le chiffrement qui suit.
13 oct. 2026	Pas de Flutter pour les applications natives ; coque Capacitor autour de l'application web.	Point D6 écarté.
14 oct. 2026	Bureau : Tauri 2 ; le paquet contient le build web tel quel.	Une seule interface à maintenir (D1).
15 oct. 2026	Commencer par Google et GitHub ; code envoyé seulement à l'inscription ; pas d'OVH, on reste sur ce serveur (un VPS plus tard) ; sans compte, les notes restent locales.	Points E2, E6 écartés ; E8 reporté.
15 oct. 2026	Synchroniser d'abord en clair, une destination par carnet (valeurs proposées, non contredites).	Le chiffrement fichier par fichier reste à faire (E16b à E16d).
16 oct. 2026	Connecteurs dans l'ordre : Nextcloud/WebDAV, GitHub, GitLab, puis Dropbox et Google Drive (OAuth).	Tous faits ; reste à les essayer pour de vrai.
16 oct. 2026	Synchronisation *par page* (un objet par page et par pièce jointe, avec un index), chiffrement *par carnet* (une clé par carnet) ; plus un carnet chiffré ou clair, jamais mixte, local ou distant, jamais mixte.	Étapes E16a à E16d ; principes ci-dessous.
16 oct. 2026	Le mot « carnet » désigne l'unité (« projet » seulement comme extension future éventuelle) ; une note est une page, et les pages peuvent avoir des sous-pages.	Point E16f reporté.
16 oct. 2026	Les carnets chiffrés réutilisent la phrase du coffre local, sans seconde phrase à taper.	Évite une demande de plus ; le coût en est expliqué à la création (E16b).

4 Ce qui dépend de vous

1. **Créer les applications OAuth** (étapes dans `docs/zim-server.md`) : Google et GitHub pour la connexion ; Dropbox et Google Drive pour les connecteurs. Tant que ce n'est pas fait, aucun bouton correspondant n'apparaît.
2. **Un service d'envoi d'e-mails** (SMTP) : sans lui, les codes d'inscription restent dans un fichier du serveur.
3. **Essayer pour de vrai** un Nextcloud, un dépôt GitHub (jeton limité à un dépôt privé), un projet GitLab, puis me dire ce qui coince : tout a été essayé seulement contre des imitations.
4. **Répondre aux décisions ouvertes** (point E18) et choisir l'éditeur légal (E19).
5. **Essayer sur de vrais appareils** : un téléphone Android, un iPhone si possible, une machine Windows.
6. À terme, **un VPS et un nom de domaine** pour ouvrir le service à d'autres (E3, E20). Aucune commande `sudo` n'est nécessaire pour l'instant.

5 Ce qui n'a jamais été vérifié

— Aucune connexion réelle à Google, GitHub, Dropbox, Google Drive, GitLab ni à un Nextcloud : les tests utilisent des imitations écrites pour eux (qui parlent le protocole tel que je le connais ; un vrai service peut répondre autrement).

— Aucun envoi réel d'e-mail, aucune charge, aucun essai avec plusieurs processus serveur.

— Aucun vrai téléphone, aucune machine Windows réelle, aucun Mac.

— Aucune relecture de sécurité par quelqu'un d'autre ; les mutants de test (on casse une protection pour voir si un test le remarque) ne remplacent pas cela.

— L'écriture dans plusieurs fichiers n'est pas atomique sur WebDAV, Dropbox et Google Drive (elle l'est sur GitHub et GitLab).

6 Journal de progression

28 septembre 2026

Échafaudage du projet, spécification du format Zim, lecteur/écrivain `zim-format` avec essais d'aller-retour (`22f83cf`, `e32b662`). Cœur Turtl intégré et remis en état (`ba24934`, `45d410a`). Plan du port wasm ; chiffrement identique au natif prouvé par un pont libsodium.js (`b505976`, `0b0b74e`).

29 septembre

Le cœur vérifie ses types pour wasm ; la vraie libsodium.js est branchée (`22f2301`, `750a02e`).

5 octobre

Port wasm : SQLite réel, crypto complète, async partout, le cœur en service dans un hôte JavaScript, persistance OPFS, essais dans un vrai Worker (`9cd33d0…7a4ce6d`). Pièces jointes (`931bc76`). Essais sur de vrais carnets Zim et fuzzer (`4439d04`). Éditeur CodeMirror 6 et commandes (`b11bf0f`, `7b3d601`).

6 octobre

Serveur statique sûr (`8a0aa09`). Règles du carnet et liaisons wasm (`4d1a1c9`, `b23f50a`). Application du carnet : arbre, liens, recherche, import/export (`8640a67`) et 31 scénarios navigateur (`00b22ec`). Pièces jointes et images (`4f5b823…55ca54e`). Comptes locaux, écritures atomiques, coffre chiffré dans l'application (`f997f29…6e5938a`).

7 octobre

https avec certificat local (`f064f6d`) ; phrase secrète normalisée (`af74b76`) ; corrections de revue (`1f9264a`, `a3dbf62`) ; application installable avec code épinglé (`cf7d045`) ; certificat de confiance (`5c89aa1`).

9 octobre

Changement de phrase, sauvegarde et restauration du coffre (`92a2ccf`, `d5985f9`, `3b3d897`). Étiquettes, journal, complétion de liens, tableaux, sommaire, calendrier (`fc66e61`, `baa6cb2`, `25522a4`).

12 et 13 octobre

Écriture sur téléphone : barre de mise en forme, tactile, clavier ; scénarios aussi en WebKit ; menu des actions repliées ; corrections de ce que montrent les vrais navigateurs (`1da565f…2a5cd9d`).

14 octobre

Coque Android (APK de débogage, essai dans une vraie WebView) (`908c50c`, `ecf48af`, `b9a7ed6`). Programme de bureau Tauri pour Windows et Linux, paquets construits en CI (`824bb06`, `afec2be`).

15 octobre

Carnets en vrais dossiers sur le bureau (`17b0a53`) ; CI plus robuste (`25c3597`, `6c33865`). Étude de faisabilité des comptes et du SaaS (`b40bdcf`). Serveur de comptes, connexion Google/GitHub, interface des comptes (`b49afd9`, `f97d995`). Stockage des carnets par compte et synchronisation dans l'application (`9693953`, `2e278ec`). Copie scellée des carnets chiffrés (`49063d6`). Connecteurs Nextcloud/WebDAV, GitHub, GitLab (`f7710c6`, `3673cac`, `eb9883d`, `74db037`).

16 octobre

Connecteurs Dropbox et Google Drive (autorisation dans la fenêtre du service, jetons chiffrés, révocation) avec leurs essais (`a0ab896` et le commit qui suit) ; boutons de dialogue qui restent visibles dans une fenêtre trop haute ; ce document de suivi. Puis cadrage du chiffrement fichier par fichier (principes et décisions ci-dessus) ; synchronisation par page des carnets en clair commencée (E16a).

7 Risques ouverts

— **Confiance dans le code servi par le web** (risque 6) : atténuée par le service worker, non résolue ; le paquet de bureau est la meilleure garantie.

— **Notes en clair sur le serveur** : tant que le chiffrement fichier par fichier (E16b à E16d) n'existent pas, un carnet « gardé sur le serveur » ou sur un connecteur est lisible par l'exploitant du service ; les fenêtres le disent.

— **Phrase secrète oubliée** : aucune récupération possible (coffre et copie scellée) ; dit avant chaque activation.

— **Dépendance à de petites imitations pour tester les services** : un vrai service peut répondre autrement ; d'où la liste de ce qui reste à essayer.

— **Fidélité à Zim** (risque 7) : l'essentiel est couvert, pas tout.

8 Comment ce document est tenu

À chaque étape terminée : (1) changer l'état et le « reste à faire » des points touchés dans `docs/suivi/suivi.tex`, ajouter les lignes nécessaires et une entrée au journal avec les identifiants de commit ; (2) mettre à jour les chiffres de tests ci-dessus (le fichier `docs/suivi/releves.tex`) ; (3) lancer `scripts/suivi.sh`, qui recompile le PDF (`docs/suivi/suivi.pdf`) ; (4) commiter le source et le PDF ensemble. Les totaux de la synthèse sont comptés automatiquement d'après les lignes du tableau.